

Levantar Logixone con IntelliJ IDEA Ultimate

Guía paso a paso para importar, configurar, construir, ejecutar y diagnosticar el ERP desde IntelliJ sin apartarse del baseline Docker.

Edición	1.0
Fecha de verificación	2026-07-30
IDE verificado	IntelliJ IDEA Ultimate 2026.2
Sistema de ejemplo	Windows 11 con PowerShell
Baseline	Java 21, Maven Wrapper 3.9.16, Docker/Compose, PostgreSQL, Keycloak 26.7.0 y WildFly 41
Distribución de demo	perfil Maven with-business-partners-demo
Edición PDF	guia-levantar-logixone-intellij-idea-ultimate.pdf

Ruta rápida: configurar Java 21 → preparar secretos → construir dos imágenes → ejecutar Compose → comprobar readiness → abrir la interfaz.

Contenido

1. Objetivo y resultado esperado	3
2. Por qué usamos Docker aunque IntelliJ Ultimate soporte WildFly	3
3. Prerrequisitos	3
4. Abrir el proyecto correctamente	4
5. Habilitar las integraciones del IDE	4
6. Configurar Java 21	4
7. Preparar una terminal de IntelliJ	5
8. Sincronizar y comprobar el reactor	5
9. Preparar el archivo local de Compose	5
10. Crear los secretos locales sin pisar los existentes	6
11. Validar la configuración antes de crear recursos	6
12. Construir las dos imágenes verificadas	7
13. Levantar el proyecto	7
14. Comprobar el estado	7
15. Administrar Docker desde IntelliJ	8
16. Ciclo diario de desarrollo	9
17. Menú y permisos al agregar plugins	9
18. Detener sin perder datos	10
19. Diagnóstico	10
20. Uso opcional de un WildFly local	11
21. Checklist final	11
22. Reversión segura	12
23. Fuentes oficiales de IntelliJ IDEA	12
24. Documentación relacionada del repositorio	12

1. Objetivo y resultado esperado

Esta guía ayuda a una persona que recibe el repositorio por primera vez a:

1. abrir correctamente el reactor Maven en IntelliJ IDEA Ultimate;
2. configurar el JDK 21 y el Maven Wrapper del proyecto;
3. preparar la configuración y los secretos locales sin versionarlos;
4. construir las imágenes verificadas de aplicación y migrador;
5. levantar PostgreSQL, migraciones, Keycloak y WildFly mediante Docker Compose;
6. comprobar la salud del sistema y abrir la interfaz;
7. detener y volver a levantar el ambiente sin perder los datos.

Al terminar, deben responder:

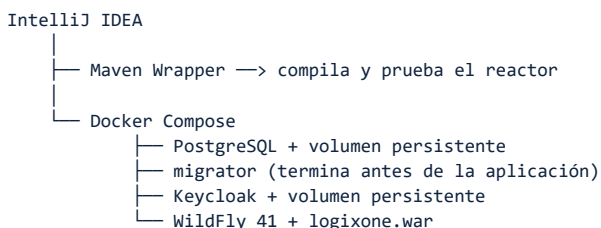
- aplicación: `http://localhost:18080/logixone/faces/app/index.xhtml`;
- administración: `http://localhost:18080/logixone/faces/admin/index.xhtml`;
- liveness: `http://localhost:18080/logixone/health/live`;
- readiness: `http://localhost:18080/logixone/health/ready`;
- Keycloak de desarrollo/demo: `http://keycloak.localhost:8180`.

Decisión operativa: IntelliJ es el entorno de edición, navegación, ejecución de pruebas y control de Docker. La forma oficial de ejecutar Logixone completo sigue siendo la imagen reproducible y `infra/compose/compose.yaml`. Instalar WildFly directamente en IntelliJ no es necesario para este primer arranque.

2. Por qué usamos Docker aunque IntelliJ Ultimate soporte WildFly

IntelliJ Ultimate puede registrar un servidor JBoss/WildFly local. Sin embargo, Logixone también necesita PostgreSQL, el migrador one-shot, Keycloak, secretos montados, configuración OIDC y la configuración reproducible de WildFly.

El recorrido oficial conserva estas piezas juntas:



Esto evita que una configuración manual del IDE funcione solamente en una computadora y difiera de la imagen que después se prueba o promueve.

3. Prerrequisitos

Antes de abrir el proyecto, comprobar:

- IntelliJ IDEA Ultimate 2026.2 instalado y con licencia activa;
- Docker Desktop iniciado y configurado para contenedores Linux;
- Docker Engine, Buildx y Docker Compose disponibles;

- repositorio ubicado en `C:\cosme\LogixoneJakarta11`;
- puertos `18080` y `8180` disponibles;
- acceso a Internet para el primer build si las dependencias o imágenes todavía no existen en las cachés locales;
- espacio suficiente para `.tools/`, las imágenes y los volúmenes Docker.

Comprobar Docker desde PowerShell:

```
docker version
docker buildx version
docker compose version
```

Los tres comandos deben terminar con código `0`. Si `docker version` muestra el cliente pero no el servidor, iniciar Docker Desktop antes de continuar.

4. Abrir el proyecto correctamente

1. Abrir IntelliJ IDEA.
2. Elegir **File > Open**.
3. Seleccionar la carpeta raíz `C:\cosme\LogixoneJakarta11`, no un submódulo ni un POM individual.
4. Confirmar **Trust Project** si IntelliJ solicita confianza.
5. Cuando detecte Maven, seleccionar **Load Maven Project**.
6. Esperar a que finalice el primer índice antes de corregir referencias.

No crear un proyecto Jakarta EE nuevo ni importar por separado `kernel`, `application` o los plugins. El `pom.xml` padre es quien define el reactor completo y sus versiones centralizadas.

5. Habilitar las integraciones del IDE

Abrir **File > Settings > Plugins > Installed** y comprobar que estén habilitadas las capacidades:

- Maven;
- Jakarta EE Platform;
- Jakarta EE: Application Servers;
- WildFly/JBoss;
- Docker.

En IntelliJ IDEA Ultimate estas integraciones se distribuyen con el IDE. Si una aparece deshabilitada, habilitarla y reiniciar IntelliJ cuando lo pida. No instalar una biblioteca visual, servidor o plugin de terceros para sustituir el baseline del repositorio.

6. Configurar Java 21

6.1 SDK del proyecto

1. Abrir **File > Project Structure > Project**.
2. En **SDK**, elegir **Add SDK > JDK**.
3. Seleccionar:
`C:\cosme\LogixoneJakarta11\.tools\jdk\jdk-21.0.11+10`
4. Nombrarlo, por ejemplo, `Logixone JDK 21.0.11`.

5. Dejar **Language level** en SDK default o Java 21.

6.2 JDK usado por Maven

Abrir **File > Settings > Build, Execution, Deployment > Build Tools > Maven**:

- **Maven home path**: Use Maven wrapper;
- **User settings file**: no forzar uno global;
- **Local repository**: dejar que `.mvn/maven.config` aplique `.tools/maven-repository`.

Luego comprobar:

- en **Maven > Importing > JDK for importer**, seleccionar el JDK 21 del proyecto;
- en **Maven > Runner > JRE**, seleccionar Project SDK.

El build canónico siempre usa `mvnw.cmd`. No usar un `mvn.exe` global ni el Maven embebido del IDE para declarar un resultado de validación.

7. Preparar una terminal de IntelliJ

Abrir **View > Tool Windows > Terminal**. Debe iniciar en la raíz del repositorio. Ejecutar en cada terminal nueva:

```
$projectRoot = (Get-Location).Path
$env:JAVA_HOME = Join-Path $projectRoot '.tools\jdk\jdk-21.0.11+10'
$env:MAVEN_USER_HOME = Join-Path $projectRoot '.tools\maven-wrapper-home'
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
.\mvnw.cmd --version
```

Resultado esperado:

- Apache Maven 3.9.16;
- Java 21.0.11;
- Maven home bajo `.tools/maven-wrapper-home`;
- dependencias del proyecto bajo `.tools/maven-repository`.

La variable se limita a la terminal actual. Así no se cambia globalmente Java para otros proyectos de la computadora.

8. Sincronizar y comprobar el reactor

En la ventana **Maven**, pulsar **Reload All Maven Projects**. Deben aparecer el padre y los módulos del reactor sin dependencias `javax.*`.

Como comprobación pequeña:

```
.\mvnw.cmd -B -pl plugin-api -am test
```

Para validar el baseline completo con el plugin de Socios Comerciales:

```
.\mvnw.cmd -B -Pwith-business-partners-demo clean verify
```

La segunda ejecución es más larga. Debe terminar con **BUILD SUCCESS**. Una prueba que se ejecute y falle se corrige antes de continuar; no se desactiva ni se omite para conseguir una compilación verde.

9. Preparar el archivo local de Compose

El archivo local no se versiona. Crearlo únicamente si todavía no existe:

```
$localEnv = 'infra\compose\compose.env.local'
if (-not (Test-Path -LiteralPath $localEnv)) {
    Copy-Item -LiteralPath 'infra\compose\compose.env.example' -Destination $localEnv
} else {
    Write-Host 'Se conserva compose.env.local existente.'
}
```

Abrir `infra/compose/compose.env.local` en IntelliJ y ajustar:

```
LOGIXONE_HTTP_PORT=18080
LOGIXONE_OIDC_REDIRECT_URI=http://localhost:18080/logixone/*
LOGIXONE_OIDC_WEB_ORIGIN=http://localhost:18080
LOGIXONE_OIDC_POST_LOGOUT_REDIRECT_URI=http://localhost:18080/logixone/faces/app/
index.xhtml
LOGIXONE_APP_IMAGE=logixone/app:j11-s6-07-closing
LOGIXONE_MIGRATOR_IMAGE=logixone/migrator:j11-s6-07-closing
```

Las cuatro direcciones públicas de aplicación deben cambiar juntas. No colocar contraseñas, tokens ni secretos dentro de este archivo.

Si ya existe un volumen de Keycloak operativo, conservar sus URL actuales. Cambiar las URI de un cliente OIDC existente exige actualizar de forma coherente su configuración; no borrar el volumen para evitar ese trabajo.

10. Crear los secretos locales sin pisar los existentes

Logixone necesita cuatro archivos externos. El siguiente bloque crea solamente los que faltan y conserva cualquier valor existente:

```
$secretDir = Join-Path (Get-Location).Path '.tools\secrets'
New-Item -ItemType Directory -Force -Path $secretDir | Out-Null

$secretNames = @(
    'postgres-password.txt',
    'keycloak-admin-password.txt',
    'oidc-client-secret.txt',
    'demo-user-password.txt'
)

foreach ($secretName in $secretNames) {
    $secretPath = Join-Path $secretDir $secretName
    if (-not (Test-Path -LiteralPath $secretPath)) {
        $bytes = New-Object byte[] 32
        $generator = [Security.Cryptography.RandomNumberGenerator]::Create()
        try {
            $generator.GetBytes($bytes)
        } finally {
            $generator.Dispose()
        }
        [IO.File]::WriteAllText(
            $secretPath,
            [Convert]::ToBase64String($bytes),
            [Text.Encoding]::ASCII
        )
        Write-Host "Creado: $secretName"
    } else {
        Write-Host "Conservado: $secretName"
    }
}
```

No abrir, imprimir, copiar al portapapeles ni registrar el contenido en una captura. El usuario de demo se llama `demo.empresas.ab`; su contraseña se consulta privadamente en `demo-user-password.txt` solo al iniciar sesión.

11. Validar la configuración antes de crear recursos

```
docker compose `
  --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml `
  config --quiet
```

Resultado esperado: código 0 y ninguna salida de error. Este comando valida la estructura sin levantar contenedores ni crear volúmenes.

12. Construir las dos imágenes verificadas

Desde la terminal de IntelliJ:

```
docker build --platform linux/amd64 `
  --build-arg LOGIXONE_BUILD_MODE=verified `
  --build-arg LOGIXONE_MAVEN_PROFILE=with-business-partners-demo `
  --tag logixone/app:j11-s6-07-closing `
  --file infra/docker/Dockerfile .
```

Después:

```
docker build --platform linux/amd64 `
  --build-arg LOGIXONE_BUILD_MODE=verified `
  --build-arg LOGIXONE_MAVEN_PROFILE=with-business-partners-demo `
  --tag logixone/migrator:j11-s6-07-closing `
  --file infra/docker/Dockerfile.migrator .
```

La primera imagen contiene WildFly 41 y el WAR. La segunda contiene el ejecutable de migraciones con el mismo conjunto físico de plugins. Usar el mismo perfil evita que la aplicación y el migrador discrepen sobre qué plugins existen.

Comprobar los tags sin imprimir configuración sensible:

```
docker image inspect logixone/app:j11-s6-07-closing `
  --format '{{.Id}}'
docker image inspect logixone/migrator:j11-s6-07-closing `
  --format '{{.Id}}'
```

13. Levantar el proyecto

```
docker compose `
  --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml `
  up -d --wait --wait-timeout 180
```

Compose realiza la secuencia:

1. crea las redes y los volúmenes nombrados si no existen;
2. inicia PostgreSQL;
3. ejecuta el migrador y espera que termine correctamente;
4. inicia Keycloak y espera su health check;
5. inicia la aplicación WildFly;
6. espera que readiness indique UP.

El migrador puede terminar con estado Exited (0). Eso es correcto: es un proceso one-shot, no un servidor permanente.

14. Comprobar el estado

14.1 Contenedores

```
docker compose `
  --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml `
  ps
```

PostgreSQL, Keycloak y app deben estar saludables. Migrator debe haber terminado con código 0.

14.2 Salud semántica

```
$live = Invoke-RestMethod `
    'http://localhost:18080/logixone/health/live'
$ready = Invoke-RestMethod `
    'http://localhost:18080/logixone/health/ready'

$live.status
$ready.status
```

Ambos deben mostrar UP. Liveness indica que el proceso puede responder; readiness comprueba además catálogo, configuración, base, migraciones y coherencia OIDC local.

14.3 Interfaz

Abrir en el navegador:

```
http://localhost:18080/logixone/faces/app/index.xhtml
```

Keycloak debe solicitar autenticación. Usar el usuario ficticio `demo.empresas.ab` y leer localmente la contraseña ya creada, sin compartirla.

15. Administrar Docker desde IntelliJ

15.1 Conexión Docker

1. Abrir **File > Settings > Build, Execution, Deployment > Docker**.
2. Agregar una conexión **Docker for Windows**.
3. Esperar el mensaje de conexión satisfactoria.
4. Abrir **View > Tool Windows > Services** para ver imágenes, contenedores, logs y estado.

15.2 Configuración Docker Compose

Se puede agregar una configuración:

1. **Run > Edit Configurations > + > Docker Compose**.
2. Nombre: Logixone - Compose.
3. Compose file: `infra/compose/compose.yaml`.
4. Environment file: `infra/compose/compose.env.local`.
5. Servicios: todos.
6. Comando: `up`.

La terminal documentada sigue siendo la referencia para una evidencia reproducible, porque muestra de forma explícita archivo, variables y timeout. La configuración visual es un atajo para el trabajo diario.

15.3 Configuración Maven útil

Crear una configuración **Maven**:

- nombre: Logixone - Verify demo;
- directorio: raíz del repositorio;
- comando: `-B -Pwith-business-partners-demo clean verify`;
- Maven: Wrapper;

- JRE: Project SDK 21.

Para depurar una prueba unitaria, abrir la clase y usar el icono de ejecución del margen. Los breakpoints funcionan sin exponer puertos de depuración del contenedor.

16. Ciclo diario de desarrollo

Después de cambiar un único módulo:

```
.\mvnw.cmd -B -pl <módulo> -am test
```

Al completar un corte coherente:

```
.\mvnw.cmd -B -Pwith-business-partners-demo clean verify
```

Si el cambio debe observarse en la aplicación:

1. reconstruir las imágenes afectadas con el mismo perfil;
2. validar Compose;
3. recrear la composición;
4. verificar health y luego la pantalla.

```
docker compose `
  --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml `
  up -d --force-recreate --wait --wait-timeout 180
```

No copiar manualmente un JAR dentro del contenedor: agregar o retirar físicamente un plugin requiere reconstruir y redespigar la distribución.

17. Menú y permisos al agregar plugins

Cuando una nueva imagen contiene otro plugin, el kernel descubre y valida sus contribuciones. El shell compone un único menú con las entradas de todos los plugins que simultáneamente:

- estén físicamente presentes;
- sean compatibles;
- estén activos para la empresa seleccionada;
- tengan sus dependencias activas;
- publiquen una entrada de menú;
- y cuyo permiso requerido tenga el usuario.

Por eso no se fusionan archivos XHTML ni menús a mano. Un plugin inactivo o un usuario sin permiso no debe ver ni poder invocar su funcionalidad.

Los permisos se administran en:

```
http://localhost:18080/logixone/faces/admin/security.xhtml
```

Esa pantalla permite administrar usuarios, membresías, roles empresariales y permisos. La autoridad global de administración se gestiona por separado en:

```
http://localhost:18080/logixone/faces/admin/system-authority.xhtml
```

Ocultar una opción no reemplaza la seguridad: cada operación vuelve a comprobar en el servidor la empresa, el plugin y el permiso efectivo.

18. Detener sin perder datos

```
docker compose `
  --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml `
  down
```

down elimina contenedores y redes recreables, pero conserva los volúmenes nombrados de PostgreSQL y Keycloak. Al ejecutar nuevamente up, Compose reutiliza esos volúmenes; el migrador reconoce las versiones y checksums ya aplicados y no recrea ni pisa los datos existentes.

No agregar --volumes. Esa opción elimina los datos de PostgreSQL y el estado local de Keycloak. Solo corresponde a un ambiente explícitamente efímero, identificado y autorizado, después de verificar el respaldo.

19. Diagnóstico

Docker no está disponible

- comprobar que Docker Desktop esté iniciado;
- revisar la conexión en **Settings > Docker**;
- ejecutar `docker version` y verificar que muestre cliente y servidor.

IntelliJ usa Java incorrecto

- revisar Project SDK, Maven Importer y Maven Runner;
- abrir una terminal nueva y volver a definir `JAVA_HOME`;
- confirmar con `.\mvnw.cmd --version`.

Maven descarga fuera de .tools

- ejecutar siempre el Wrapper del repositorio;
- definir `MAVEN_USER_HOME` antes del primer uso de esa terminal;
- confirmar que `.mvn/maven.config` conserva `-Dmaven.repo.local=.tools/maven-repository`.

Compose informa variables o secretos ausentes

- comprobar que existe `infra/compose/compose.env.local`;
- comprobar solo los nombres de archivo bajo `.tools/secrets/`;
- no imprimir sus contenidos ni copiarlos al archivo `.env`.

Migrator falla

```
docker compose `
  --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml `
  logs --no-color postgres migrator
```

Corregir imagen, configuración o migración. No editar a mano las tablas de historial ni cambiar una migración ya aplicada.

Liveness está UP pero readiness está DOWN

```
docker compose `
  --env-file infra/compose/compose.env.local `
  -f infra/compose/compose.yaml `
  logs --no-color postgres migrator keycloak app
```

Revisar el componente marcado DOWN en readiness. Es normal que liveness siga verde cuando una dependencia impide atender tráfico.

La aplicación redirige mal después de cambiar el puerto

Comprobar juntas LOGIXONE_HTTP_PORT, LOGIXONE_OIDC_REDIRECT_URI, LOGIXONE_OIDC_WEB_ORIGIN y LOGIXONE_OIDC_POST_LOGOUT_REDIRECT_URI. Un realm ya creado conserva su cliente; no eliminar su volumen como primera solución.

Falta una opción del menú

Comprobar, en este orden:

1. que la imagen contenga el plugin;
2. que el catálogo sea válido;
3. que el plugin esté activo para la empresa;
4. que sus dependencias estén activas;
5. que el usuario tenga membresía y permiso;
6. que se haya seleccionado la empresa correcta.

Logs completos

```
docker compose `
--env-file infra/compose/compose.env.local `
-f infra/compose/compose.yaml `
logs --no-color postgres migrator keycloak app
```

No adjuntar volcados de variables, tokens ni contenidos de /run/secrets.

20. Uso opcional de un WildFly local

IntelliJ Ultimate 2026.2 permite crear una configuración **JBoss/WildFly Server**. Esa integración sirve para despliegue o depuración local, pero el repositorio no entrega actualmente una configuración local equivalente a la imagen oficial: datasource, driver, CLI, OIDC, estabilidad preview, secretos y migraciones deben coincidir exactamente.

Por lo tanto:

- no es necesaria para levantar el proyecto;
- no sustituye Docker/Compose en pruebas, demo o evidencia;
- no debe configurarse improvisando credenciales o habilitando actualización automática de Hibernate;
- solo debe adoptarse cuando exista un runbook versionado que reproduzca el baseline completo.

21. Checklist final

- [] Abrí la raíz del reactor Maven.
- [] Project SDK, Maven Importer y Maven Runner usan Java 21.
- [] Maven usa el Wrapper y las cachés del proyecto bajo .tools.
- [] Docker Desktop responde desde IntelliJ y PowerShell.
- [] compose.env.local existe y no contiene secretos.
- [] Los cuatro secretos existen y no fueron sobrescritos.

- [] Compose valida con código 0.
- [] Las imágenes app y migrator usan el mismo perfil.
- [] Migrator terminó con código 0.
- [] PostgreSQL, Keycloak y app están saludables.
- [] Liveness y readiness responden UP.
- [] La interfaz abre y redirige a Keycloak.
- [] Sé detener con down sin eliminar volúmenes.

22. Reversión segura

Si el arranque no debe continuar:

1. guardar logs sin secretos;
2. ejecutar `docker compose ... down` sin `--volumes`;
3. conservar `compose.env.local`, los cuatro secretos y ambos volúmenes;
4. corregir la causa;
5. reconstruir una imagen identificada y repetir validación, migrador y health.

No usar `git reset --hard`, no borrar `.tools`, no eliminar volúmenes y no modificar la base manualmente para ocultar el fallo.

23. Fuentes oficiales de IntelliJ IDEA

Consultadas el 2026-07-30:

- [IntelliJ IDEA 2026.2 — publicación y correcciones](#)
- [Importar un proyecto o módulo](#)
- [Configuración de importación Maven](#)
- [Soporte Maven](#)
- [Integración con servidores de aplicación](#)
- [Configuración JBoss/WildFly](#)
- [Configuraciones Docker y Docker Compose](#)

24. Documentación relacionada del repositorio

- [Construcción local con Maven Wrapper](#)
- [Construcción de imágenes](#)
- [Operación de Compose](#)
- [Migrador](#)
- [Keycloak y OIDC](#)
- [Demo visual de cierre de Sprint 6](#)
- [Guía de implementación por empresa](#)